



EmergenTheta: Experimental Analyses within the Theta Framework (Competition Contribution)

Milán Mondok[✉], Csanád Telbisz^{*}, Levente Bajczi[✉],
Dániel Kovács[✉], Mihály Dobos-Kovács[✉], and Vince Molnár[✉]

Department of Artificial Intelligence and Systems Engineering
Budapest University of Technology and Economics, Budapest, Hungary
`mondok@mit.bme.hu`

Abstract. EMERGENTHETA is our sandbox for emerging analyses within the THETA framework. Previously established analyses have been migrated to THETA, and EMERGENTHETA focuses exclusively on novelties for both counterexample-guided abstraction refinement over control-flow automata and reachability analyses over symbolic transition systems. The 2026 submission presents a combined predicate-explicit abstraction domain, multithreaded symbolic transition system encodings, on-the-fly reachability and counterexample generation for generalized saturation, and an IC3 backend. This is the first year that we were able to submit EMERGENTHETA to all categories, representing a significant step in the evolution of THETA, and the competition results indicate the practical viability of the new techniques.

1 Verification Approach

The algorithms of EMERGENTHETA fall into two categories: 1) the *counterexample-guided abstraction refinement* (CEGAR) algorithm that conducts explicit step-by-step exploration of the abstract state space by building an abstract reachability graph as described in [11], which operates directly on the control-flow automaton (CFA) description created from the C program, and 2) various reachability algorithms that use the low-level symbolic transition system (STS) description created from the CFA. The latter category contains the *generalized saturation* (GSAT) [12,14], the *property-directed reachability* (IC3/PDR) [10] and *bounded model checking* (BMC) [7] algorithms. See Figure 1 for an overview, presenting novelties exclusive to EMERGENTHETA (not used by the mainline THETA).

Novelties related to the CEGAR algorithm. The *combined abstraction domain* [13] is a combination of the predicate and explicit-value abstraction domains, which tracks a subset of the variables explicitly, while also tracking the truth-values of a set of Boolean predicates. States are pairs of explicit-value and

* Jury member representing EMERGENTHETA at SV-COMP 2026.

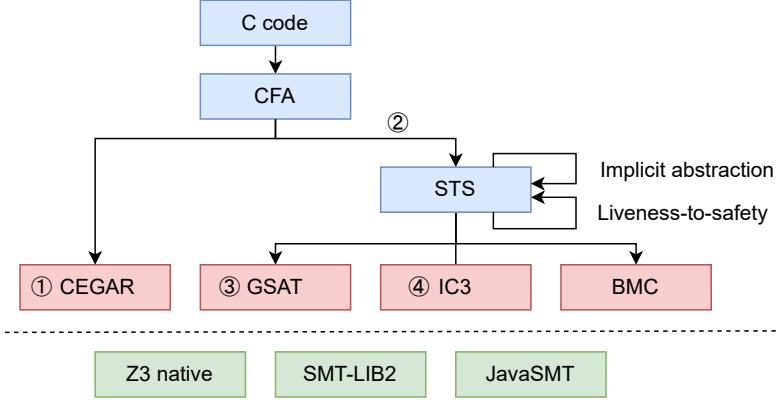


Fig. 1: Overview of the verification approach of EMERGENTHETA. 2026 additions are ① *combined domain* for CEGAR, ② *multithreaded CFA-to-STS transformation*, ③ *on-the-fly reachability and counterexample generation* for the generalized saturation algorithm, ④ an *IC3* algorithm backend. Blue, red and green rectangles respectively denote *task representations*, *algorithm backends* and *SMT-solver backends*.

predicate states, the transfer function reasons about both domains simultaneously. By default all variables are tracked through predicates and the algorithm uses heuristics to decide if a variable should be tracked explicitly instead.

A preprocessing step collects all literal expressions co-occurring with each variable (in the CFA, this means assumptions and assignments involving both the variable and the literal). During refinement, if a new predicate introduces previously unseen literals with the variable, we switch to explicit tracking. This helps detect “predicate counting”, which typically occurs when the solver returns interpolants containing literals absent from the model.

Novelties related to STS-based algorithms. This class of algorithms operates on *symbolic transition systems* (STS), which encode reachability problems using three SMT formulae (the initial states, the transition relation, and the safety invariant). Before verification, we translate the CFA representation into an STS, encoding control locations as variables. As a recent extension, this transformation supports *multithreaded programs* by statically determining an upper bound on the number of threads and encoding each thread’s control location with a separate variable. Programs that may spawn an unbounded number of threads cannot be encoded finitely; while array-based SMT encodings are possible, we found them inefficient. In such cases, we encode only a finite number of threads and report unknown if no bug is found and threads may be created unboundedly. This novel transformation is only used by EMERGENTHETA and not the mainline THETA.

Generalized saturation [12] utilizes substitution diagrams [14] to dynamically create multi-valued decision diagrams from the SMT descriptions of the model, enabling the use of the symbolic algorithm that was originally defined on Petri nets. However, saturation performs enumeration of the entire state space and cannot return counterexamples. A novel addition to the algorithm this year is *on-the-fly reachability*, which we conduct using a modification of constrained saturation that traverses the set of target states simultaneously with the transition relation and stops enumeration whenever a target state is reached. A further addition is *counterexample generation*: we use depth-first search to find a counterexample in the intersection of 1) states from which the target states can be reached, and 2) the states explored during on-the-fly reachability. Furthermore, the multithreaded CFA-to-STS transformation allowed us to use generalized saturation on multithreaded C programs this year, where we expect it to perform well, as it is designed to exploit symmetries in asynchronous behaviour.

A new addition to the set of algorithms this year is *property directed reachability* (IC3) [10], which analyzes the model incrementally through the proof of several lemmas, eventually forming an inductive invariant proving the system correct or directing the search towards a counterexample.

2 Software Architecture

EMERGENTHETA is a JVM-based tool, implemented in Kotlin, with legacy components written in Java. EMERGENTHETA is part of the THETA framework [11], and can therefore use its integrated frontend- and solver infrastructure. Parsing C files is done using a custom ANTLR grammar [2]. SMT solvers can be used in three ways: Z3 [16] natively via its Java API, other solvers (e.g., MathSAT5 [8], CVC5 [3]) via SMT-LIBv2 [9], and JavaSMT [1]. For SV-COMP’26, we use Z3 4.5.0 and 4.14.0; MathSAT5 5.6.12; and CVC5 1.2.0.

For 2026, we completely reworked the transformation engine that manages model transformations to handle the wide number of input formalisms, algorithm backends and model transformations flexibly using a pipeline architecture with active passes. We provide support for *overflow*, *memory safety*, and *data race* properties via transformations that modify the control-flow automata. We also provide support for *termination* properties through a *liveness-to-safety* [6] transformation, implemented as an STS-to-STS transformation. These transformations represent a significant step in the architecture of THETA, but are not unique to EMERGENTHETA.

For SV-COMP, we combine multiple configurations in sequential portfolios and select the appropriate portfolio based on traits of the C program (e.g., presence of arrays, multithreading, floating point operations). Each configuration is executed with a predefined timeout and we switch to a fallback SMT solver in case of a solver error. For multithreaded programs, the portfolio applies GSAT, BMC, and combined CEGAR in this order. For single-threaded programs, the portfolio uses combined CEGAR using multiple refinement strategies, followed by GSAT and IC3, both augmented with implicit predicate abstraction [17]. For

Configuration	no-data-race		no-overflow		termination		unreach-call		memsafety	
	safe	unsafe	safe	unsafe	safe	unsafe	safe	unsafe	safe	unsafe
GSAT	557	32	163	0	0	0	90	124	191	0
Combined CEGAR	5	10	392	214	0	0	1136	364	204	12
IC3	0	0	0	3	0	0	0	2	0	0
BMC	7	9	8	3	0	0	10	16	16	0
THETA fallback	1	2	191	15	0	0	674	122	15	2
Liveness-CEGAR	0	0	0	0	268	113	0	0	0	0
Liveness-to-safety	0	0	0	0	16	11	0	0	0	0
$\sum$ Correct	570	53	754	235	284	124	1910	628	426	14
$\sum$ Wrong	0	1	1	12	0	0	5	2	2	13
$\sum$ Unconfirmed correct	0	14	0	33	0	0	118	23	0	6

Table 1: Verification results per configuration and property

termination tasks, the portfolio begins with liveness-CEGAR [15], followed by BMC and k-induction using the liveness-to-safety [6] transformation. Portfolios fall back to the stable portfolio (`--portfolio STABLE`) if all configurations fail.

3 Strengths and Weaknesses of the Approach

Table 1 presents our results [4] per property (*memsafety* combines all properties related to memory safety). The combined CEGAR and the GSAT algorithms managed to solve the most tasks partly due to the fact that they are placed at the start of the portfolios (explaining the relatively low number of solved tasks by the IMC and BMC algorithms, which were placed further behind in the portfolios). GSAT performed very promisingly on multithreaded problems, and was able to return counterexamples for the *unreach-call* property. Using liveness-CEGAR, we were able to solve more *termination* tasks this year, than last year using the transformation of TransVer [5] (408 vs 217). EMERGENTHETA produced a few wrong results (36 in total), the main reason behind which are parsing issues in the frontend of our tool. Overall, EMERGENTHETA solved 4998 tasks this year, achieving an overall score of 11248.

4 Software Project, Tool Setup and Configuration

EMERGENTHETA is integrated into the THETA verification framework maintained by the Critical Systems Research Group¹ of the Budapest University of Technology and Economics. Using the competition archive [18] for software verification, EMERGENTHETA can be started using the following input options: `./theta-start.sh <input> --svcomp --portfolio EMERGENT`. This command starts the EMERGENT portfolio that we used at SV-COMP 2026. This portfolio handles all categories of C verification at SV-COMP using our emerging configuration options. For the first time this year, we participate in all categories.

¹ <https://ftsrg.mit.bme.hu/en/>

5 Data-Availability Statement

The project is available open-source on GitHub² under an Apache 2.0 license, and the binary release of the competition version (6.27.9) is published on Zenodo [18].

Funding. This research was partially funded by the EKÖP-25-3 New National Excellence Program under project numbers EKÖP-25-3-BME-243, and EKÖP-25-3-BME-22, and the Doctoral Excellence Fellowship Programme under project numbers 400434/2023 and 400443/2023; funded by the NRDI Fund of Hungary.

References

1. Baier, D., Beyer, D., Friedberger, K.: JavaSMT 3: Interacting with SMT Solvers in Java. In: Silva, A., Leino, K.R.M. (eds.) *Computer Aided Verification. Lecture Notes in Computer Science*, vol. 12760, pp. 195–208. Springer International Publishing, Cham (2021). https://doi.org/10.1007/978-3-030-81688-9_9
2. Bajcsi, L., Ádám, Z., Molnár, V.: C for Yourself: Comparison of Front-End Techniques for Formal Verification. In: *10th FormaliSEICSE 2022*. pp. 1–11. ACM (2022). <https://doi.org/10.1145/3524482.3527646>
3. Barbosa, H., Barrett, C., Brain, M., Kremer, G., Lachnitt, H., Mann, M., Mohamed, A., Mohamed, M., Niemetz, A., Nötzli, A., Ozdemir, A., Preiner, M., Reynolds, A., Sheng, Y., Tinelli, C., Zohar, Y.: cvc5: A Versatile and Industrial-Strength SMT Solver. In: Fisman, D., Rosu, G. (eds.) *TACAS 2022*. pp. 415–442. Springer International Publishing, Cham (2022). https://doi.org/10.1007/978-3-030-99524-9_24
4. Beyer, D., Strejček, J.: Evaluating software verifiers for C, Java, and SV-LIB (report on SV-COMP 2026). In: *Proc. TACAS (2)*. LNCS 16506, Springer (2026)
5. Beyer, D., Jankola, M., Lingsch-Rosenfeld, M., Xia, T., Zheng, X.: A modular program-transformation framework for reducing specifications to reachability (2025), <https://arxiv.org/abs/2501.16310>
6. Biere, A., Artho, C., Schuppan, V.: Liveness checking as safety checking. In: *FMICS 2002, ICALP 2002 Satellite Workshop (2002)*. [https://doi.org/10.1016/S1571-0661\(04\)80410-9](https://doi.org/10.1016/S1571-0661(04)80410-9)
7. Biere, A., Cimatti, A., Clarke, E., Zhu, Y.: Symbolic Model Checking without BDDs. In: *TACAS (1999)*. https://doi.org/10.1007/3-540-49059-0_14
8. Cimatti, A., Griggio, A., Schaafsma, B., Sebastiani, R.: The MathSAT5 SMT Solver. In: *TACAS 2013, LNCS*, vol. 7795, pp. 93–107. Springer (2013). https://doi.org/10.1007/978-3-642-36742-7_7
9. Dobos-Kovács, M., Vörös, A.: Evaluation of SMT solvers in abstraction-based software model checking. In: *Proceedings of the 11th Latin-American Symposium on Dependable Computing*. p. 109–116. LADC '22, Association for Computing Machinery, New York, NY, USA (2023). <https://doi.org/10.1145/3569902.3570187>
10. Een, N., Mishchenko, A., Brayton, R.: Efficient implementation of property directed reachability. In: *Proceedings of the International Conference on Formal Methods in Computer-Aided Design*. p. 125–134. FMCAD '11, FMCAD Inc, Austin, Texas (2011), <https://dl.acm.org/doi/10.5555/2157654.2157675>

² <https://github.com/ftsrg/theta/releases/tag/svcomp26>

11. Hajdu, Á., Micskei, Z.: Efficient Strategies for CEGAR-based Model Checking. *Journal of Automated Reasoning* **64**(6), 1051–1091 (2020). <https://doi.org/10.1007/s10817-019-09535-x>
12. Molnár, V., Majzik, I.: Saturation Enhanced with Conditional Locality: Application to Petri Nets. In: Donatelli, S., Haar, S. (eds.) *Application and Theory of Petri Nets and Concurrency*. pp. 342–361. Springer International Publishing, Cham (2019)
13. Mondok, M.: Efficient formal verification of component-based engineering models. Master's thesis, Budapest University of Technology and Economics (2022)
14. Mondok, M., Molnár, V.: Efficient Manipulation of Logical Formulas as Decision Diagrams. In: Renczes, B. (ed.) *Proceedings of the 31st PhD Mini-Symposium*. pp. 61–65. Budapest University of Technology and Economics, Department of Measurement and Information Systems (2024). <https://doi.org/10.3311/MINISY2024-012>
15. Mondok, M., Vörös, A.: Abstraction-based model checking of linear temporal properties. In: Renczes, B. (ed.) *Proceedings of the 27th PhD Mini-Symposium*. pp. 29–32. Budapest University of Technology and Economics, Department of Measurement and Information Systems (2020)
16. de Moura, L., Bjørner, N.: Z3: An Efficient SMT Solver. In: *TACAS 2008, LNCS*, vol. 4963, pp. 337–340. Springer (2008). https://doi.org/10.1007/978-3-540-78800-3_24
17. Tonetta, S.: Abstract Model Checking without Computing the Abstraction. In: Cavalcanti, A., Dams, D.R. (eds.) *FM 2009: Formal Methods*. pp. 89–105. Springer Berlin Heidelberg, Berlin, Heidelberg (2009)
18. Ádám, Z., Bajcsi, L., Dobos-Kovács, M., Molnár, V., Mondok, M., Somorjai, M., Szekeres, D., Telbisz, C., Kovács, D., Rippl, B.R.: Emergentheta - verifier archive (Dec 2025). <https://doi.org/10.5281/zenodo.17779912>

Open Access. This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (<http://creativecommons.org/licenses/by/4.0/>), which permits use, sharing, adaptation, distribution, and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

